

Understanding Inference Scaling for LLMs: Bottlenecks, Trade-offs, and Performance Principles

Moiz Arif*, Avinash Maurya[†], Sudharshan Vazhkudai*, Bogdan Nicolae[†]

^{*}Micron Technology Inc., Austin, TX, USA

[†]Argonne National Laboratory, Lemont, IL, USA

Email: ^{*}{marifa, svazhkudai}@micron.com; [†]{amaurya, bnicolae}@anl.gov

Abstract—The transition from standard generative AI to *reasoning-centric architectures*, exemplified by models capable of extensive Chain-of-Thought (CoT) processing, marks a fundamental paradigm shift in system requirements. Unlike traditional workloads dominated by compute-bound prefill, reasoning workloads generate long chains of reasoning tokens that shift inference into a *Capacity-Bound regime*. This paper presents a comprehensive system characterization, evaluating models ranging from 8B to 671B parameters on GPU clusters. By systematically exploring the interplay between Data, Tensor, and Pipeline parallelism, we identify critical bottlenecks that defy standard scaling heuristics. Our analysis reveals that data parallelism is throughput efficient for small models but hits a capacity trap on reasoning workloads as KV-cache fragmentation forces early throttling resulting in sub-optimal compute utilization. Tensor parallelism unlocks stranded memory and delivers sublinear gains near the 32B crossover. At frontier scale, dense models (e.g., Llama-405B) are interconnect and memory-bandwidth bound and favor high-degree TP, while sparse Mixture-of-Experts (MoE) models (e.g., DeepSeek-R1) are limited by routing and synchronization latency and benefit from hybrid strategies. These insights provide a rigorous decision framework for navigating the reasoning cliff, establishing new architectural imperatives for the next generation of inference infrastructure.

Index Terms—LLM inference, parallelism and scheduling Analysis, performance characterization, memory capacity wall

I. INTRODUCTION

A. Motivation

The deployment of Large Language Models (LLMs) has transitioned from a capability demonstration to a cornerstone of industrial computing. However, the emergence of *reasoning-centric architectures*, exemplified by DeepSeek-R1 [16] and OpenAI’s o1 [28] marks a fundamental paradigm shift in system requirements. Unlike standard generative models that prioritize fluent text generation, reasoning models employ CoT [35] like processing to generate extensive intermediate logic traces before producing a final answer.

This shift fundamentally alters the resource signature of AI inference. While traditional LLM serving is often dominated by compute-bound prefill or bandwidth-bound decoding of short responses, reasoning workloads typically demonstrate a *Capacity-Bound* inference regime. The generation of thousands of intermediate “thinking” tokens [34] creates massive, persistent Key-Value (KV) cache footprints that saturate High Bandwidth Memory (HBM) long before compute utilization peaks [37]. Consequently, standard scaling heuristics such as

maximizing batch size for throughput or relying solely on Data Parallelism (DP) become counter-productive, triggering severe memory thrashing, scheduler preemption, and non-linear latency spikes [1], [23].

Deploying these frontier-scale models (often exceeding 400B parameters [18], [25]) requires navigating a complex, high-dimensional design space [21]. System architects must balance the low-latency requirements of interactive reasoning against the strict capacity limits of modern accelerators. This necessitates a move beyond monolithic scaling toward nuanced combinations of Data, Tensor [33], and Pipeline parallelism [20], tailored specifically to the sparsity and sequence length characteristics of the model [39].

B. System Challenges in Reasoning Inference

Deploying reasoning-centric models at scale introduces distinct system-level bottlenecks that are structurally different from training [6] or standard chat inference:

- **The Capacity Wall:** Reasoning traces with Output Sequence Length (OSL) $\gg 10k$ cause the KV cache to grow linearly, rapidly exhausting per-GPU HBM [24]. In DP with engines such as vLLM [22], this leads to “stranded capacity” where fragmented memory across replicas forces premature request throttling despite low compute utilization [37].
- **The Parallelism Efficiency Gap:** While Tensor Parallelism (TP) alleviates memory pressure by using collective HBM of multiple GPUs, it incurs communication overheads that degrade performance beyond intra-node scaling [10]. Conversely, Pipeline Parallelism (PP) offers memory relief but suffers from “pipeline bubbles” that are difficult to hide in serial, auto-regressive generation tasks [3].
- **Architectural Divergence:** The optimal strategy is no longer universal as we show in our analysis. As we demonstrate, dense models (e.g., Llama-3.1-405B) behave fundamentally differently from sparse MoE models (e.g., DeepSeek-R1-671B), requiring bespoke parallelization hierarchies to align with their respective compute-to-communication ratios [8].

While recent work has improved kernel efficiency and KV management, the dominant scaling limits for reasoning workloads increasingly arise from system-level capacity and scheduling dynamics rather than operator inefficiencies. This work isolates and quantifies these limits under realistic inference conditions.

C. Contributions

This paper presents a systematic characterization of GPU-based inference for reasoning-centric LLMs. We evaluate models ranging from 8B to 671B parameters on a cluster of NVIDIA H200 GPUs, identifying the breakpoints where standard scaling laws fall short. Our specific contributions are:

- 1) **Characterization of the “Parallelism Transition Point”:** We empirically determine the model size and sequence length thresholds where DP collapses due to KV saturation, necessitating a transition to hybrid or TP approaches.
- 2) **Quantification of the “Reasoning Gap”:** We demonstrate how the extended decoding phases of reasoning models shift the critical bottleneck from prefill compute (TTFT) to decode memory capacity (TPOT).
- 3) **Evaluation of Frontier Architectures:** We provide a comparative analysis of large dense versus sparse scaling, revealing that MoE architectures favor hybrid strategies (PP+TP) to mitigate synchronization costs, whereas dense models demand high-bandwidth TP configurations.
- 4) **Operational Guidance:** Based on these insights, we provide guidance for selecting optimal parallelism strategies that maximize fleet-level throughput while meeting strict latency SLAs for reasoning tasks.

II. BACKGROUND AND RELATED WORK

A. Reasoning Inference Breakdown

Reasoning-centric inference is distinct from standard chat workloads in its execution flow and resource demands. It proceeds in two phases with orthogonal hardware requirements:

Prefill (The Compute Phase): The model processes the user prompt (input sequence) in parallel and dominated by large matrix-matrix multiplications (GEMMs) and is strictly compute-bound. Modern GPUs effectively utilize their tensor cores, achieving high Streaming Multiprocessor (SM) occupancy. The latency of this phase determines the TTFT.

Decode (The Memory Phase): Following prefill, the model enters the autoregressive generation loop. Unlike standard chat ($OSL \approx 500$), reasoning traces can exceed 10000 tokens [36]. Each token generation requires reading the entire model weight set and the active KV cache from HBM [37]. This phase is strictly *bandwidth-bound*; the arithmetic intensity collapses, and the GPU spends the majority of cycles performing memory reads. Our telemetry indicates that for reasoning workloads, the system spends $> 99\%$ of its wall-clock time in this inefficient regime [2].

B. Model Architectures and Attention Mechanisms

The impact of the GPU memory exhaustion is modulated by the specific architecture of the model. We characterize two distinct paradigms found in state-of-the-art reasoning engines:

Dense Architectures (Grouped-Query Attention): Models like *Llama-3.1-405B* [18] utilize a dense transformer architecture where every parameter is active for every token. To mitigate memory pressure, these models employ *Grouped-Query Attention (GQA)* (typically 8 KV heads) [4], which

reduces the KV footprint by $3\times-8\times$ compared to standard Multi-Head Attention (MHA) [12]. However, the memory cost remains linear with layer count [5]. For the 405B model (≈ 126 layers), the KV cache consumes ≈ 1.05 MB per token in FP16. Serving a batch of 128 requests with 10k reasoning tokens each consumes over 1.3 TB of memory solely for the cache, far outstripping the capacity of a single H200 GPU.

Sparse Architectures (Multi-Head Latent Attention): The *DeepSeek-R1-671B* [16] model utilizes a MoE architecture, activating only $\approx 37B$ parameters per token. Crucially, it employs *Multi-Head Latent Attention (MLA)*, which compresses the KV cache into a low-rank latent vector. This architectural choice effectively decouples the KV cache size from the number of attention heads, allowing R1 to sustain long reasoning contexts with a relatively lower memory footprint than dense models of same scale [26].

C. Memory Hierarchy and Scheduling

To manage these massive footprints, we utilize *vLLM* [22], an advanced, widely-used inference engine with *PagedAttention*, which partitions the KV cache into non-contiguous blocks to eliminate internal fragmentation. The scheduler plays a critical role in this architecture:

- **Chunked Prefill:** To maximize GPU throughput, the scheduler splits long input sequences into smaller chunks [3]. These chunks are processed iteratively and added greedily to batches, allowing the system to interleave prefill computation with ongoing decode requests. This smooths SM occupancy and mitigates head-of-line blocking, though it can fragment memory traffic.
- **Preemption:** When the “Reasoning Cliff” is reached (HBM saturation), the scheduler must preempt active requests [37], moving them to a “Waiting” queue or swapping them to CPU host memory. This is a defensive mechanism to prevent OOM but triggers a re-computation or swapping penalty that degrades tail latency.

We focus on HBM-resident KV caches to isolate fundamental capacity and bandwidth limits of reasoning-centric inference without introducing latency trade-offs from disaggregated memory tiers. Techniques such as KV offloading [27], prefetching [17], quantization [11], and compression expand the optimization space but are ultimately constrained along the decode path by latency, bandwidth, and scheduling overheads. We therefore treat these techniques as complementary and orthogonal to our analysis, which characterizes when KV-cache pressure dominates even under HBM-only conditions.

D. Parallelism Taxonomy

Scaling these models requires distributing computation across several GPUs. We evaluate four strategies:

Data Parallelism (DP): The model is replicated on each GPU. While efficient for throughput, DP suffers from *KV Fragmentation* as each replica holds a full copy of weights (e.g., $\approx 800GB$ for 405B which exceeds single GPU memory capacity and even for smaller models, weight replication reduces available KV space).

Tensor Parallelism (TP): Individual layers are sharded across GPUs. TP uses the aggregated HBM capacity without unnecessary model replication. However, it introduces high-frequency all-Reduce communication at every layer [1]. For the 32B model, we observe a transition where the benefit of aggregate memory outperforms the cost of communication.

Pipeline Parallelism (PP): Layers are partitioned sequentially across GPUs. PP reduces memory footprint without the high communication frequency of TP. However, it introduces “Pipeline Bubbles” (idle time) [20] due to lack of sufficient concurrent requests to fill these bubbles without incurring unacceptable queueing latency.

Hybrid Parallelism: A hierarchical combination (e.g., TP within a node, PP across nodes). Our analysis suggests this is critical for frontier sparse models to balance memory pooling with synchronization overheads.

III. EXPERIMENTAL METHODOLOGY

A. Testbed and Inference Engine

All experiments are performed on a high-performance inference node equipped with $8 \times$ NVIDIA H200 Tensor Core GPUs (SXM5 form factor). Each H200 GPU features 141 GB of HBM3e memory providing a peak memory bandwidth of 4.8 TB/s, and delivers a theoretical peak of 1,979 TFLOPS for FP16/BF16 tensor operations. The GPUs are fully interconnected via fourth-generation NVLink and NVSwitch, providing a bidirectional GPU-to-GPU bandwidth of 900 GB/s per GPU, facilitating efficient all-reduce operations essential for TP. The host system is driven by dual Intel Xeon Platinum 8558P processors with 2TB DDR5 system memory.

We employ the vLLM v1 inference engine, leveraging its PagedAttention mechanism to mitigate memory fragmentation. The engine is configured with a block size of $B = 16$ to balance quantization granularity with memory access efficiency. We utilize the default First-Come-First-Served (FCFS) scheduling policy but tune the `max_num_batched_tokens` and `max_num_seqs` parameters to characterize the concurrency limits.

Our evaluation focuses on system-level behavior within a single NVLink-connected 8-GPU node, which serves as the fundamental scaling unit for modern LLM inference. While kernel-level optimizations and fine-grained overlap of communication and computation can improve absolute efficiency, the per-replica constraints imposed by KV-cache capacity, memory bandwidth, and scheduler dynamics under long-context reasoning workloads remain the same. Larger deployments primarily scale throughput via DP replication of this node-level behavior, with TP confined to the NVLink domain. Accordingly, we abstract away micro-architectural kernel specialization to quantify when system-level effects dominate and common scaling heuristics break down.

B. Dataset Characterization

To evaluate the “Reasoning Shift”, we utilize Meta’s Natural Reasoning dataset [38], comprised of 1.15 million multi-hop

reasoning and commonsense inference samples. Unlike standard chat workloads dominated by input processing (prefill), this dataset forces models into a generation-heavy regime. Figure 1 illustrates the divergent token length distributions that characterize “reasoning” versus traditional generative LLM workloads. Specifically, it compares input sequence lengths (ISL) versus output sequence lengths (OSL), highlighting that the majority of prompts remain relatively short while their outputs become exceptionally long. A thorough analysis of 100k random samples uncovers a distinct workload profile:

- **Input Sequence Length (ISL):** 77% of prompts are 50–150 tokens, and very few exceed 300 tokens (Figure 1a).
- **Output Sequence Length (OSL):** 45% of responses exceed 5000 tokens, indicating not only larger outputs but also complex reasoning chains (Figure 1b).
- **Reasoning Density:** Similar to OSL, reasoning steps are highly verbose, with 43.04% of responses contain over 5000 “reasoning tokens” reflecting detailed multi-step reasoning token generation (Figure 1c).

Architecturally, these distributions underscore a key stress point: even modest numbers of reasoning requests can saturate GPU memory capacity due to massive KV cache footprints from long outputs, well before reaching full compute utilization. This shifts the paradigm from prefill compute density to decode memory bandwidth/capacity and interconnect latency, providing critical motivation for treating memory capacity as a first-class design parameter—since conventional focus on FLOPs/bandwidth alone overlooks this token explosion effect inherent to reasoning workloads.

C. Model Selection

We evaluate a spectrum of models representing both the current dense architecture and the emerging reasoning-centric sparse architectures.

Small-Scale Reasoning (Distilled): We utilize DeepSeek-R1-Distill-Llama/Qwen variants (Llama-8B, Qwen-14B, Qwen-32B, Llama-70B). These models are architecturally dense but fine-tuned to output long chain-of-thought traces, allowing us to test the capacity trap of running heavy reasoning workloads on commodity-class parameters. These models utilize Grouped-Query Attention (GQA) to reduce the KV footprint by $3 \times - 8 \times$ compared to the classical Multi-Head Attention (MHA) but the memory cost remains linear with layer count. For instance, the 32B model (≈ 64 layers) consumes ≈ 262 KB/token in FP16, whereas the 70B model (≈ 80 layers) reaches ≈ 328 KB/token.

Frontier-Scale Baselines: We consider the case of Llama-3.1-405B, a standard dense transformer serving as the heavy-compute baseline. With ≈ 126 layers and dense activation, it exerts maximum pressure on HBM bandwidth. Its KV footprint is massive (≈ 1.05 MB/token in FP16), necessitating aggressive quantization or paging for long-context inference. Next, we consider DeepSeek-R1-671B, a MoE model with ≈ 37 B active parameters per token utilizing Multi-Head Latent Attention (MLA), which compresses the KV cache into a low-rank latent vector (smaller than GQA). This architectural

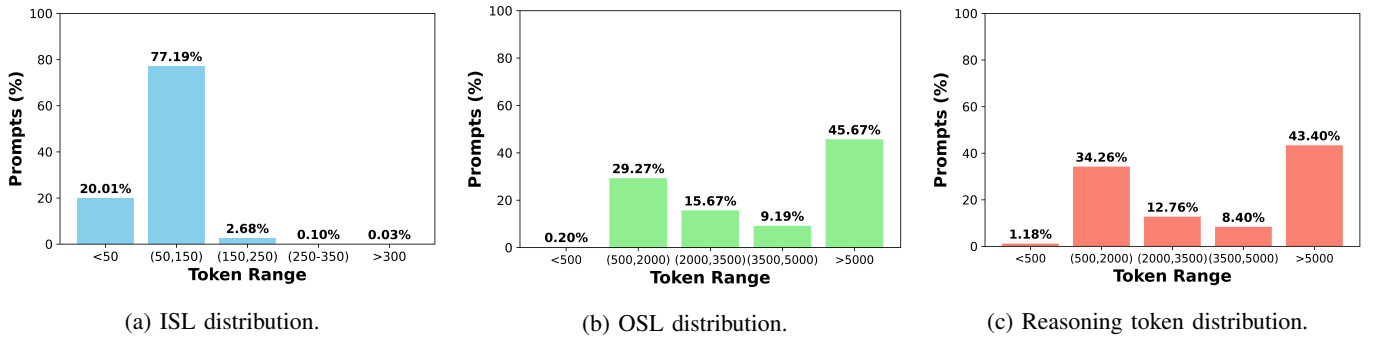


Fig. 1: Input, output, and reasoning token distributions for 100k samples from Meta’s Natural Reasoning dataset.

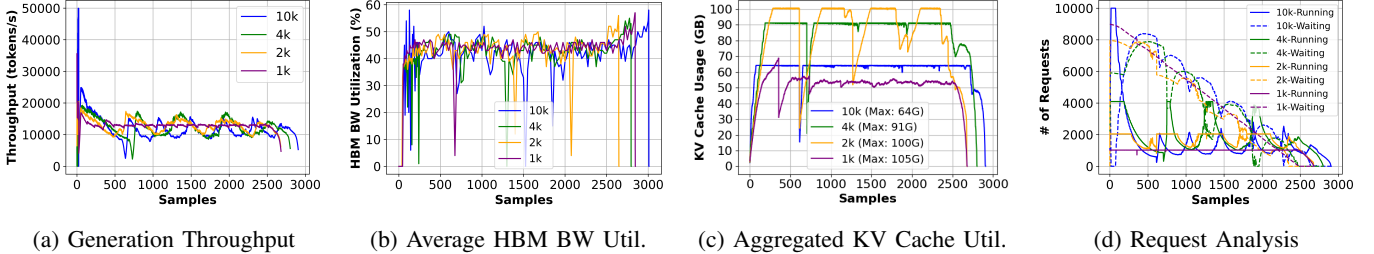


Fig. 2: Timeline of inference engine metrics on scaling the number of sequences for DeepSeek-8B on one H200 GPU.

choice decouples KV size from the number of attention heads, allowing R1 to sustain long reasoning contexts with a significantly lower memory footprint per generated token compared to a dense model of equivalent scale.

D. Profiling Methodology

We employ a multi-layered instrumentation strategy, capturing high-level service metrics via the inference engine and low-level resource telemetry via hardware counters. This dual-view approach allows us to correlate end-to-end latency artifacts with specific micro-architectural bottlenecks.

- **Time-To-First-Token (TTFT):** The latency from request arrival to the generation of the first token. In reasoning workloads, TTFT is dominated by the prefill phase and queueing delays. High TTFT indicates prefill compute saturation or head-of-line blocking [31] caused by long-running decode phases of prior requests.
- **Time-Per-Output-Token (TPOT):** The average inter-token latency during the generation phase. This metric is a direct proxy for memory bandwidth efficiency during autoregressive decoding. Increases in TPOT signal HBM bandwidth saturation or excessive communication overhead in tensor-parallel configurations.
- **Generation Throughput:** The aggregate number of tokens generated per second across all GPUs. This system-level metric captures the efficacy of batching; sublinear scaling of throughput with batch size reveals the concurrency wall where memory capacity limits active slots.
- **End-to-End (E2E) Latency:** The total elapsed time from request submission to the completion of the final token. This metric aggregates queueing delays, prefill compute, and the extended decoding phase.

- **Request Lifecycle Tracking:** We trace the state transitions of individual requests to decompose latency into Waiting (queueing) and Running (execution) components. This granular tracking allows us to isolate scheduler-induced delays, such as preemption or admission throttling caused by KV-cache fragmentation, from hardware execution time.
- **GPU/HBM Bandwidth Utilization:** Measured via `nvidia-smi` to track GPU utilization and memory bandwidth.
- **KV-Cache Saturation:** We monitor the KV cache utilization against the total allocated KV cache. This metric is the critical indicator of the capacity trap; nearing 100% saturation forces the scheduler to preempt requests to free memory, causing catastrophic spikes in end-to-end latency due to re-computation costs.

Together, these metrics allow us to distinguish compute saturation, memory bandwidth saturation, and KV-capacity-driven preemption effects, enabling causal attribution of end-to-end latency degradation under scaling.

IV. ANALYSIS I: CAPACITY TRAP FOR SMALL MODELS

This section characterizes the performance boundaries of small-sized models (8B–32B) when serving memory-intensive reasoning workloads. We investigate the “What-If” scenario: *Does maximizing GPU occupancy via high concurrency yield sustainable throughput for reasoning, or does it trigger a resource collapse?*

A. The Concurrency-Capacity Trade-off

In traditional inference, increasing the maximum number of concurrent sequences (`max_num_seqs`) is the primary lever to amortize kernel launch overheads and hide HBM latency. We evaluate this heuristic with DeepSeek-8B model for a

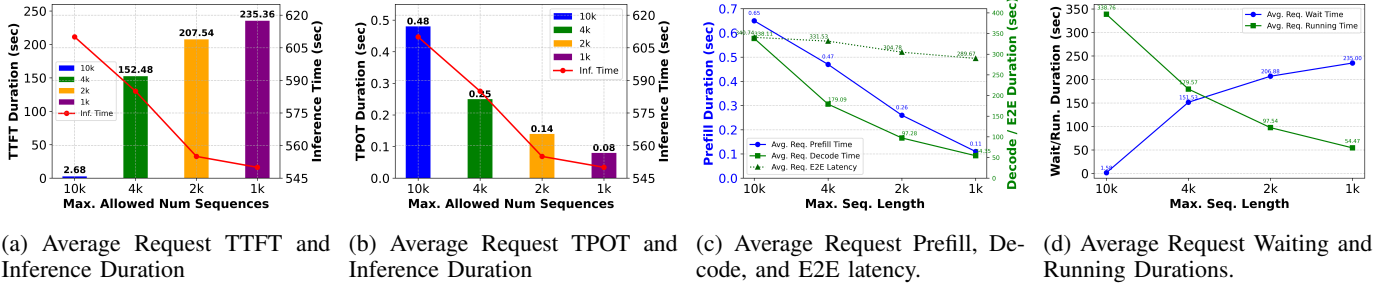


Fig. 3: Overall serving statistics on scaling maximum number of sequences for DeepSeek-8B on one H200 GPU.

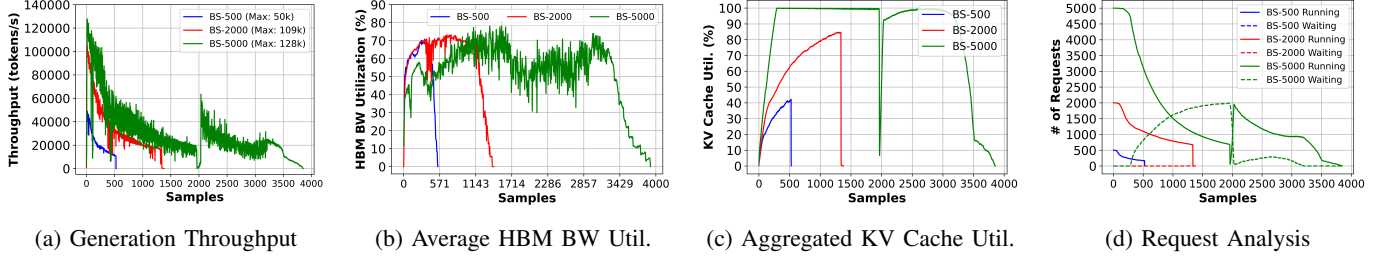


Fig. 4: Batch size scaling for DeepSeek-8B on 8x H200 GPUs with 8-way DP.

batch of 10K input sequences while scaling concurrency with ‘max_num_seqs’ from 1K to 10K on one H200 GPU.

The Occupancy-Saturation Conflict: Our max-sequence scaling experiments (Figure 2) reveal a fundamental conflict between compute utilization and memory availability. Figure 2 illustrates the temporal evolution of throughput, bandwidth utilization, KV occupancy, and request state as concurrency scales. While higher concurrency initially increases throughput (Figure 2a), the corresponding rise in KV utilization (Figure 2c) rapidly saturates HBM, triggering scheduler preemption events (Figure 2d), which in turn cause drops in bandwidth utilization ((Figure 2b)). The x-axis samples represent uniformly spaced measurements and are proportional to elapsed time. Increasing concurrent sequences to 10K (Figure 2a) achieves the highest *initial* throughput because the scheduler can immediately saturate the SMs with prefill tokens. However, this advantage is transient due to limited memory capacity. Similarly, the bandwidth utilization (Figure 2b) closely follows the generation throughput trend, while dropping during memory saturation and re-scheduling.

The Preemption Cliff: As shown in Figure 2c, the 10K configuration drives the aggregated KV usage to 100% almost instantly. Unlike smaller batch settings (1K), which maintain a stable memory footprint, the 10K setting forces the vLLM scheduler into a thrashing regime. To prevent OOM errors, the scheduler must preempt active (Running) requests (Figure 2d), demoting them to the Waiting queue.

The Re-computation Penalty: This thrashing introduces a severe latency penalty. When preempted requests are rescheduled, the engine attempts to recover their state via *prefix caching*. However, under memory exhaustion, typically, the prefix match fails, and the system falls back to full prefill re-computation [37]. Even with successful partial matching, the overhead of searching blocks (either GPU-resident cached

by vLLM or offloaded prefixes cached through LMCache [9], MoonCake [29], [30], etc.) and partial recomputations destroys tail latency stability.

Analytical KV Sizing: While KV-cache capacity can be analytically estimated from model and batching parameters to guide admission control, our results show that such estimates are insufficient to prevent preemptions under dynamic, long-context reasoning workloads, where scheduling effects and KV fragmentation create transient capacity spikes.

Observation 1: For reasoning workloads, increasing concurrency improves GPU occupancy only until the cumulative KV footprint from long-lived reasoning tokens saturates HBM creating a Capacity Trap. Beyond this point, additional requests trigger preemption and re-computation, causing throughput gains to collapse. Hence, schedulers should avoid maximizing concurrency blindly; instead, enforce a KV-aware concurrency cap based on available HBM headroom, active sequence length, and expected decode growth.

B. The Latency Decoupling: TTFT vs. TPOT

While the Capacity Trap explains *why* the system struggles, the impact on service quality is best understood by studying the request TTFT, TPOT, and E2E. Figure 3 quantifies this Pareto frontier. Importantly, the convex E2E latency curve reflects not a tuning artifact, but a fundamental trade-off between admission latency and KV-limited decode progress, defining a concurrency sweet spot intrinsic to reasoning workloads. We define inference time as the total execution time of prefill and decode after request scheduling, while the E2E latency captures the full request lifecycle, including queuing, scheduling, and inference.

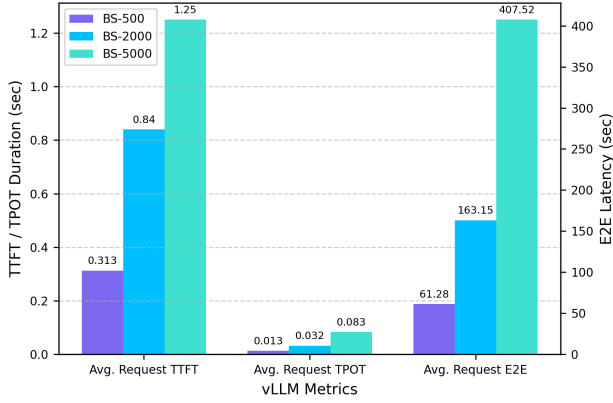


Fig. 5: vLLM Metrics for 500, 2000 and 5000 batch sizes for DS-8B for 8-way DP.

The Inverse Scaling Law: We observe a sharp divergence in metric behavior as concurrency scales: (1) **TTFT (Queue-Bound):** TTFT is minimized at maximum concurrency (10K), dropping to $\approx 2.7s$ (Figure 3a). This is intuitive: with more open slots, a request spends less time in the admission queue. High concurrency more efficiently utilizes GPU’s parallelization capabilities; (2) **TPOT (Bandwidth and Capacity Bound):** Conversely, TPOT degrades linearly with concurrency, rising from $\approx 0.08s$ at 1K to $\approx 0.48s$ at 10K (Figure 3b). In addition to being bandwidth-bound (frequent reads and limited writes per iteration), the decode phase is also capacity bound. For an increasing number of requests, preemptions due to memory constraints worsen TPOT.

The End-to-End Convexity: The End-to-End (E2E) latency (Figure 3c) reveals the net effect of these opposing forces and exhibits a non-monotonic convexity with a distinct sweet spot at the concurrency level of $\approx 2K$ sequences. This 2K point provides the best latency–capacity trade-off as it captures most of the latency improvement from reducing concurrency from 10K to 1K, while still preserving twice the sequence capacity of the 1K setting. Reducing further to 1K yields diminishing returns relative to the loss in context capacity, as reflected by higher TTFT (Figure 3a) and increased request wait time (Figure 3d), despite only marginal capacity gains. In contrast, beyond 2K, capacity degrades rapidly due to increasing decode KV pressure (Figure 3a–3d). We also agree that the observed batch-E2E behavior is sub-linear and will correct this characterization.

- **Low Concurrency Regime (<2K):** E2E latency is dominated by *Queueing Delay*. Although the GPU processes active requests fast (low TPOT), new requests wait too long to enter the system (Figure 3d) indicating insufficient in-flight work to keep the system saturated, resulting in poor throughput efficiency despite low per-request latency.
- **High Concurrency Regime (>2K):** E2E latency is dominated by *Execution Dilution* and *Preemption*. The active requests run slowly (high TPOT) due to capacity and bandwidth contention, and the tail latency spikes due to the preemption artifacts described in Observation 1. New

requests enter the system the fastest but spend longer durations in the running state (Figure 3d).

Observation 2: Reasoning workloads expose a direct trade-off between admission latency (TTFT) and generation latency (TPOT). Larger batches reduce queuing delay and improve TTFT, but they also reduce per-request available memory capacity and bandwidth, increasing TPOT during long decode phases. The optimal operating point is therefore the batch size where TTFT reduction no longer compensates for TPOT degradation. This motivates online batch-size tuning using TTFT, TPOT, KV occupancy, and HBM bandwidth as feedback signals.

C. Batch Size Scaling on 8 GPUs

To counter the effects reduced memory capacity and bandwidth during the decode phase, we use DP to scale the workload to 8 GPUs. Intuitively, DP would alleviate the pressure by distributing the requests across independent model replicas. We investigate this by fixing the cluster size ($8 \times H200$, $DP=8$) and scaling the input **Batch Size (BS)** from 500 to 5000 requests (Figure 4).

Persisting DP Memory Saturation: Despite the cluster having ≈ 1.1 TB of aggregate HBM, Figure 4c shows that the “Capacity Trap” persists at scale (e.g., BS-5000) while lower load shows under-utilized KV cache. Specifically: (1) **Throughput vs. Latency Divergence:** Scaling from BS-500 to BS-5000 increases aggregate throughput (Figure 4a). However, Figure 5 shows that End-to-End (E2E) latency grows sub-linearly ($61s \rightarrow 165s$). Similarly, the TTFT and TPOT also increase with increase load on the compute cluster while observing request throttling for BS-5000 due to fully utilized KV cache (Figure 4d); (2) **Why DP Fails to buffer Capacity:** At BS-5000, each of the 8 GPUs is assigned ≈ 625 requests. Since DP does not pool memory, each GPU effectively operates as an isolated island facing the same overload scenario characterized in Observation 1. The HBM bandwidth utilization (Figure 4b) saturates but exhibits dips corresponding to scheduler thrashing, proving that adding GPUs via DP scales *compute* but does not resolve the per-stream *memory pressure*, which we investigate next.

Observation 3: Data Parallelism allows the system to admit more requests across replicas, but each GPU still stores a full copy of the model and independently faces the same KV capacity limit. Thus, DP improves aggregate serving capacity only when each replica remains below its local HBM saturation point. For reasoning-heavy workloads, DP should be combined with admission control or memory-aware routing to prevent each replica from independently entering a preemption-heavy regime.

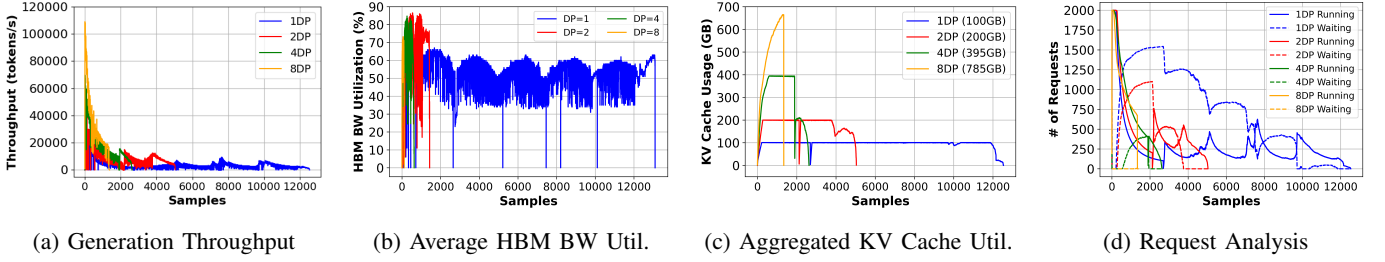


Fig. 6: Scale up for DeepSeek-8B model (best strategy: DP scaling).

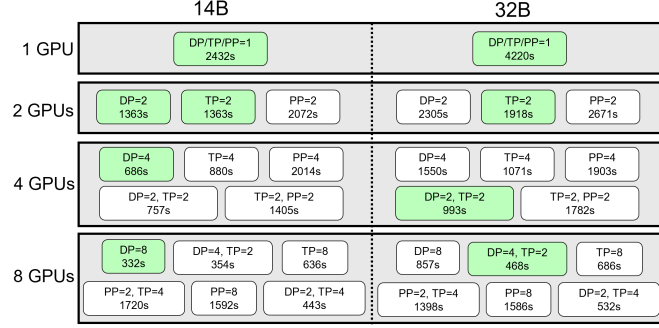


Fig. 7: Mixed config scaling for small models (2k BS).

D. The Limits of Data Parallelism (DP)

Data Parallelism (DP) is often the default scaling strategy as it avoids inter-GPU communication. We investigate its efficacy by scaling the 8B model from 1 to 8 GPUs (Figure 6). This setup provides an idealized view of near-linear throughput scaling under independent replicas. However, it also exposes inefficiencies arising from request skew and memory imbalance, which limit achievable gains in practice.

Throughput Linearity vs. Resource Efficiency: Figure 6a confirms that DP achieves near-linear scaling in aggregate throughput (4 $\times$ gains from 2 to 8 GPUs). However, this masks a critical inefficiency: the system achieves throughput by processing more streams of requests in parallel on different DP replicas, not by accelerating the throughput of any request.

- **Stranded Capacity:** DP relies on a shared-nothing memory architecture, where a request on GPU 0 cannot access free memory on GPU 1. This creates “stranded capacity”, where one replica thrashes (triggering preemption) while another has idle pages. The imbalance worsens under skewed request arrival patterns and dynamic KV growth, leading to underutilized HBM and degraded throughput/latency efficiency across the cluster.
- **Bandwidth Interference:** Figure 6b shows HBM bandwidth oscillating violently (40%–85%) rather than staying saturated. This “sawtooth” pattern represents the forced interleaving of compute-bound prefills (from the waiting queue) with memory-bound decodes. Because DP replicas are capacity-constrained (Figure 6c), they must constantly ingest new requests to fill voids left by completed ones, causing resource contention that destabilizes tail latency. The capacity contention triggers request preemption (Figure 6d) with lower degrees of DP experiencing throttling.

Observation 4: DP replicates model weights and partitions requests rather than pooling memory, and does not increase the KV capacity available to an individual long-running request. For reasoning workloads with long and variable chain-of-thought lengths, tail latency is therefore dominated by the replica that reaches KV saturation first. This indicates that DP-only scaling is insufficient when performance is limited by per-request KV growth; memory pooling, TP, or KV offload is required to increase the effective capacity margin.

V. ANALYSIS II: 3D PARALLELISM FOR LARGE MODELS

While Analysis I (§ IV) characterized the “Capacity Trap” inherent to DP, this section investigates the “What-If” scenario: *Do alternative parallelism strategies like TP or PP offer acceleration remedies, or do they introduce new bottlenecks as models scale?* We trace the scaling efficacy from 14B to 671B parameters to uncover divergences in optimal strategies.

A. From DP to TP Scaling

To determine where DP fails, we compare its scaling efficiency against Tensor Parallelism (TP) for small-to-medium models (Figure 9 vs. Figure 8) with batch size of 2K requests.

DP vs TP Inflection Point: For the 8B and 14B models, TP introduces communication overhead that outweighs its benefits. However, the 32B model reveals a critical architectural inflection point:

- **Sublinear DP vs. higher-efficiency TP:** As shown in Figure 8, DP scaling for the 32B model yields diminishing returns, achieving a 4.9 $\times$ speedup on 8 GPUs. In contrast, Figure 9 shows that TP achieves a higher, though still sublinear, speedup of 6.15 $\times$ by reducing inference duration from 4219s to 686s.
- **Why TP Wins (The Capacity Release):** The 32B model weights consume ≈ 64 GB in FP16. In a DP configuration, every GPU must replicate this 64 GB, leaving only ≈ 77 GB of the 141 GB HBM for KV cache. This creates a hard capacity ceiling that forces preemption. In contrast, TP shards the weights; at TP=8, the model consumes only ≈ 8 GB per GPU, freeing up ≈ 133 GB per device for KV cache. This massive release of HBM capacity allows TP to sustain a much larger active batch size without preemption, eliminating the re-computation overhead that throttles DP.

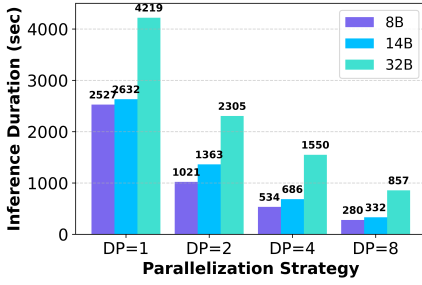


Fig. 8: DP Scaling for small models.

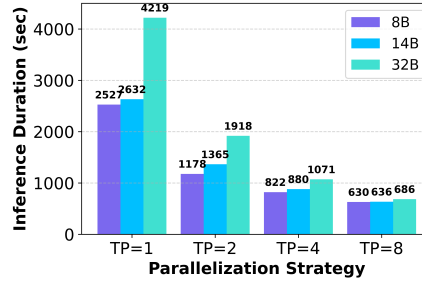


Fig. 9: TP Scaling for small models.

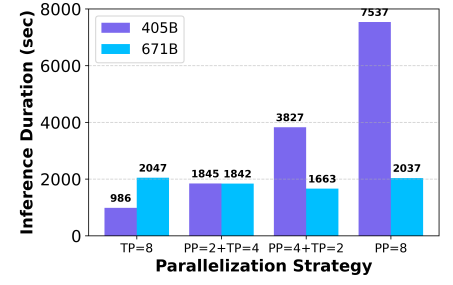
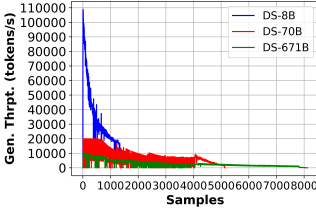
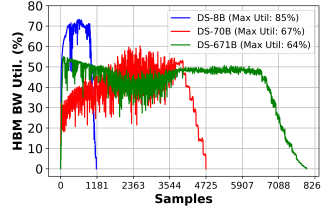


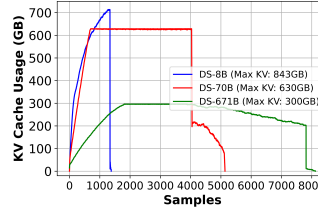
Fig. 10: E2E for large models (2k BS).



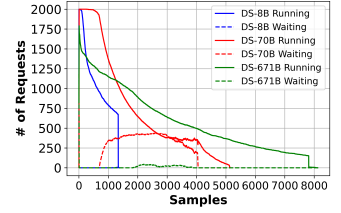
(a) Generation Throughput



(b) Average HBM BW Util.



(c) Aggregated KV Cache Util.



(d) Request Analysis

Fig. 11: Model parameter scaling on 8x H200 GPUs.

These results indicate that the benefit of TP arises primarily from released KV capacity rather than improved kernel efficiency, as the performance inflection aligns with the point at which DP replicas become KV-capacity-bound. This suggests that memory headroom, not compute efficiency, is the dominant scaling bottleneck, and that TP’s gains stem from alleviating KV pressure rather than accelerating execution.

B. Hybrid Parallelism Sweet Spot for Small Models

Figure 7 explores the trade-off space of hybrid configurations (DP+TP, TP+PP) for 14B and 32B models on a fixed 8-GPU budget. It highlights how combining parallelism strategies balances communication overhead and HBM efficiency, revealing regimes where hybrid schemes outperform pure DP or TP by reducing preemption while maintaining scalable throughput.

Balancing Communication Overhead vs. Memory Capacity: The data reveals that optimal performance requires balancing per-device latency (TP) with cluster-level concurrency (DP).

- **14B (DP Dominant):** The optimal configuration is pure DP=8 (332s). Hybrid strategies like $PP = 2 + TP = 4$ (1172s) are $\approx 3.5\times$ slower. The model is too small to justify the synchronization costs of model parallelism; simply running more independent streams is most efficient.
- **32B (Hybrid Dominant):** The optimal configuration is neither pure DP (857s) nor pure TP (686s), but the hybrid **DP=4 + TP=2** (484s). This configuration uses minimal TP ($TP = 2$) to alleviate the per-device capacity/bandwidth bottleneck to prevent preemption, while dedicating the remaining scaling factor ($DP = 4$) to concurrency. This “Right-Sized” TP approach outperforms pure TP by

$\approx 30\%$, proving that for medium models, minimizing communication degree is as critical as maximizing bandwidth.

Observation 5: The preferred parallelism strategy depends on whether the dominant overhead is communication or capacity-induced preemption. Smaller models, such as 14B, favor DP because their weights fit comfortably within per-GPU HBM and TP communication overhead is not justified. For larger models, such as 32B, TP becomes beneficial because sharding model weights releases HBM for KV cache. Once the cost of DP preemption and recomputation exceeds TP’s NVLink communication overhead, TP becomes the better scaling strategy.

C. Frontier Scaling: Dense vs. MoE Divergence

Scaling to frontier-class models (405B and 671B) requires adaptation of “Parallelism” based on model architecture. As shown in Figure 10, there is no single universal strategy; instead, the optimal approach varies based on model density.

Llama-405B (Dense): For the dense Llama-405B, due to large KV footprint the combined memory capacity of all GPUs is needed limiting DP=1. TP=8 or PP=8 are the only viable strategies, with TP=8 completing in 986s, while PP=8 is catastrophic (7537s).

- **The Penalty of Density:** Llama-405B activates all parameters for every token. This creates a massive computational workload that should theoretically hide pipeline bubbles. However, the $7.6\times$ slowdown of PP suggests the opposite. The massive KV state of 405B (1.05 MB/token) restricts the number of micro-batches that can fit in memory, forcing

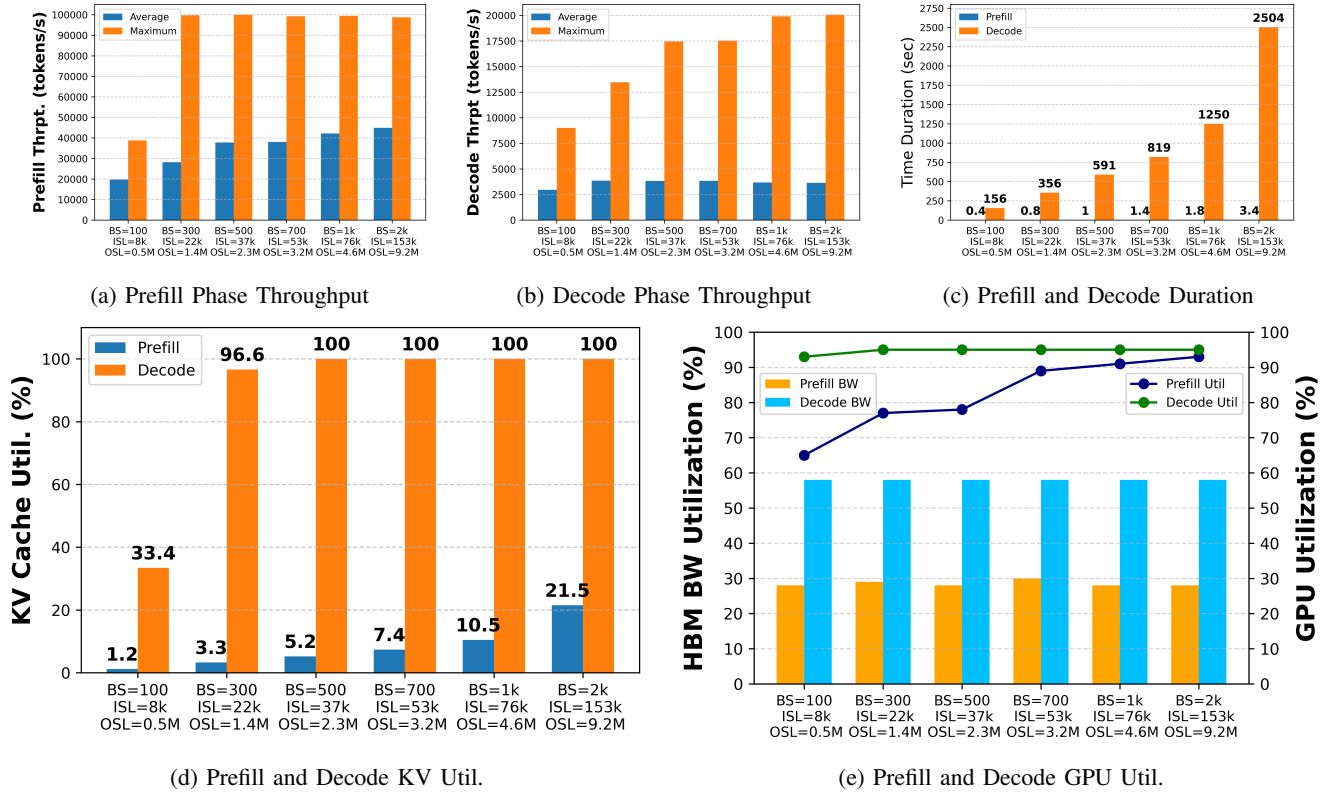


Fig. 12: Analysis of Prefill and Decode Phase during AI Inference.

the pipeline to run with large bubbles (idle time). Despite high arithmetic intensity, dense models such as Llama-405B cannot amortize pipeline bubbles due to insufficient KV capacity per stage, causing PP to exacerbate rather than alleviate decode-phase stalls.

- **Inter-Stage Traffic:** Furthermore, moving the dense activation tensors between PP stages saturates the interconnect. The TP=8 configuration avoids this by keeping activations node-local and only exchanging reduced gradients/activations over NVLink, which offers higher bandwidth than the PCIe/Link paths often used for PP stage transitions.

DeepSeek-R1-671B (MoE): In stark contrast, the sparse DeepSeek-R1 model favors a Hybrid PP=4 + TP=2 strategy (1663s), beating pure TP=8 (2047s).

- **Sensitivity to Synchronization:** While R1 is larger (671B), it is a MoE model with only ≈ 37 B active parameters. This low active parameter count means the compute-to-communication ratio is much lower than 405B. In a TP=8 setup, the constant all-reduce synchronization becomes a bottleneck because the GPU spends less time computing between syncs.
- **The MLA Advantage:** R1 employs MLA, which significantly compresses KV cache. This architectural feature works synergistically with PP. The reduced memory footprint allows R1 to support a higher micro-batch depth than 405B, effectively filling the pipeline bubbles.

- **Optimal Balance:** With $PP = 4$, R1 splits the model into smaller stages that fit in memory, and by using $TP = 2$, it minimizes pipeline bubble overhead while using combined GPU capacity. This result (1663s vs 2047s) confirms that for sparse reasoning models, minimizing TP degree is preferred over aggregating capacity and bandwidth.

Observation 6: Dense models benefit from higher-degree TP when inference is limited by HBM bandwidth and model-state footprint, since TP aggregates both memory capacity and bandwidth across GPUs. In contrast, sparse MoE models can become more sensitive to synchronization and pipeline imbalance; therefore, hybrid strategies with higher PP and lower TP may be preferable when TP communication dominates. This suggests that serving systems should choose DP/TP/PP configurations using model-specific profiling rather than a fixed parallelism policy.

D. Impact of Scaling Model Parameters

To isolate the impact of model scale on inference dynamics, we evaluate three distinct model classes—DeepSeek-8B (Small/Dense), DeepSeek-70B (Medium/Dense), and DeepSeek-R1-671B (Frontier/Sparse)—on a fixed hardware budget of $8 \times \text{H200}$ GPUs. Each model uses its optimal parallelization strategy: pure DP for 8B, Hybrid TP for 70B, and Hybrid PP+TP for 671B.

The Sublinear Throughput Degradation: Figure 11a illustrates the generation throughput. As expected, the peak throughput drops as parameter count increases, but the degradation is notably *sublinear*. A $9\times$ parameter increase from 8B to 70B results in only a $5\times$ – $6\times$ drop in peak throughput. This efficiency gain is driven by Tensor Parallelism (used for 70B), which aggregates memory bandwidth across GPUs, partially offsetting the increased FLOPs requirement. However, the 671B model (green curve) exhibits a long, flat tail, reflecting the extended “reasoning” nature of its output compared to the bursty completion of the smaller distilled models.

The Bandwidth-Compute Inversion: Telemetry reveals a fundamental shift in bottlenecks as models scale:

- **Small Models (8B):** Exhibit the highest HBM utilization ($\approx 85\%$, Figure 11b). The workload is memory-bandwidth bound because the small weight matrices are loaded rapidly, saturating the bus.
- **Frontier Models (671B):** Despite their size, they show *lower* average HBM utilization (≈ 50 – 60%). This counter-intuitive result confirms that ultra-large sparse models are bound by **synchronization and routing latency**, not raw memory bandwidth. The overhead of Pipeline Parallelism bubbles and MoE expert routing prevents the system from saturating the HBM links.

The MLA Anomaly in Capacity: Figure 11c highlights a critical architectural advantage of DeepSeek-R1. Despite having $10\times$ the parameters of the 70B model, its rate of KV-cache consumption is surprisingly moderate. This is due to Multi-Head Latent Attention (MLA), which compresses the KV state. In contrast, the dense 70B model (red line) aggressively consumes capacity, reaching its ceiling faster relative to its throughput resulting in request throttling (Figure 11d). This suggests that for future reasoning systems, architectural compression (like MLA) is as critical as hardware capacity for sustaining long-context inference.

VI. ANALYSIS III: PREFILL VS DECODE RESOURCE REQUIREMENT DIVERGENCE

The preceding analyses established that reasoning workloads are capacity-constrained (Analysis I) and sensitive to parallelism overheads (Analysis II). This section investigates the underlying physical cause: the extreme *resource divergence* between the Prefill and Decode phases. We characterize the “What-If” scenario: *Can a monolithic accelerator architecture efficiently serve a workload that oscillates between two orthogonal hardware bottlenecks?*

A. Compute vs Memory Bound Phases

Inference is often treated as a single workload, but telemetry from the 8B model (Figure 12) and frontier models (Figure 13) reveals that the system effectively behaves as two distinct machines. In this series of experiments, we vary the batch size from 100 to 2000 requests to observe how the resource profile evolves. Figures 12–15 collectively trace how resource utilization shifts across inference phases and model scales, revealing that reasoning workloads operate almost entirely in

a decode-dominated regime where memory bandwidth and KV capacity, rather than compute, determine the inference throughput and latency.

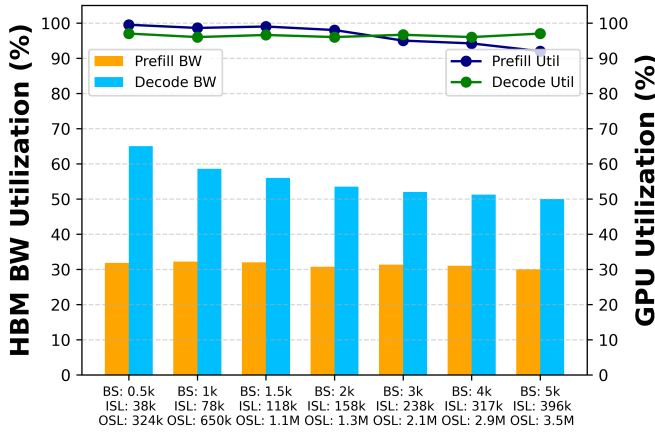
The Compute-Bound Prefill: During the prefill phase, the engine processes all input tokens in parallel. Figure 12a shows high prefill throughput for varying batch sizes showcasing the increase in average throughput with increasing context, while Figure 12e reveals the corresponding resource signature: SM Occupancy is high, but HBM bandwidth utilization remains low ($\approx 30\%$) for the 8B model and $\approx 20\%$ for the larger 405B and 671B models (Figure 13b). This indicates that prefill is *Compute-Bound*. The arithmetic intensity (FLOPs/Byte) is high because the matrix-matrix multiplications (GEMMs) can reuse loaded weights across all tokens in the prompt. In this phase, the H200’s 4.8 TB/s bandwidth is underutilized and the KV footprint (Figure 12d) also staying lower and increasing moderately for higher context, effectively leaving performance on the table. Lower prefill utilization (Figure 12e) for 8B model at small batches arises because prefill is memory/synchronization-limited rather than compute-bound. Frequent kernel launches and cross-GPU synchronization reduce sustained SM occupancy, while short prefill duration (Figure 12c), minimal KV usage (Figure 12d), and moderate HBM bandwidth (Figure 12e) indicate stall-dominated execution. Decode operates in a steady-state with fewer synchronizations/better locality, leading to higher utilization. As ISL increases, prefill utilization improves due to higher arithmetic intensity from MLP/attention computations.

The Bandwidth-Bound Decode: The transition to the decode phase inverts this profile as shown in Figure 12b showcasing lower throughput compared to prefill but shows a rising trend with increasing context. Figure 12e shows high HBM bandwidth saturation ($\approx 85\%$ for 8B, $\approx 65\%$ for 405B in Figure 13a), while SM occupancy drops or becomes variable. This phase is *Memory-Bound*. The arithmetic intensity collapses because the auto-regressive generation requires loading the entire model and KV cache to generate a single token. For reasoning workloads where $OSL \gg ISL$, the system spends the vast majority of wall-clock time in this inefficient, bandwidth-limited regime.

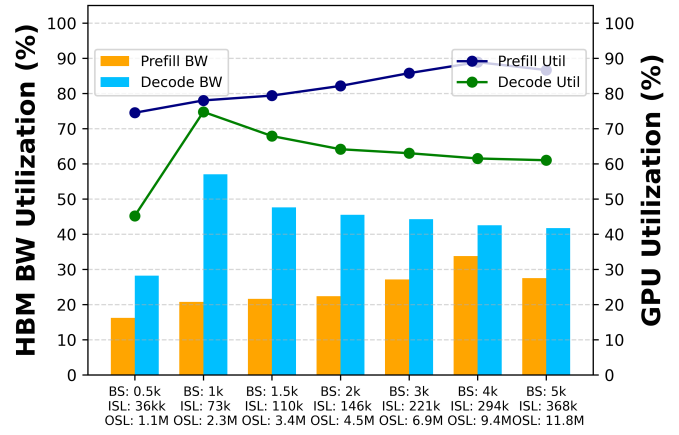
Observation 7: Reasoning workloads increase the fraction of time spent in decode, where arithmetic intensity is lower than prefill and performance is more constrained by KV movement and HBM bandwidth. As a result, high-FLOP compute units can remain underutilized even while latency remains high. This suggests that reasoning-serving optimization should prioritize KV locality, bandwidth efficiency, and decode scheduling rather than only maximizing peak compute utilization.

B. The Reasoning Cliff: KV Scaling

The severity of the memory bottleneck is dictated by the growth of the KV cache relative to the model weights. We

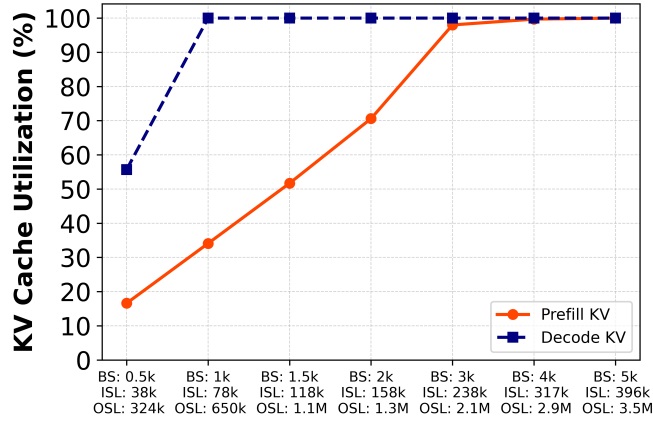


(a) GPU/HBM utilization for Llama405B.

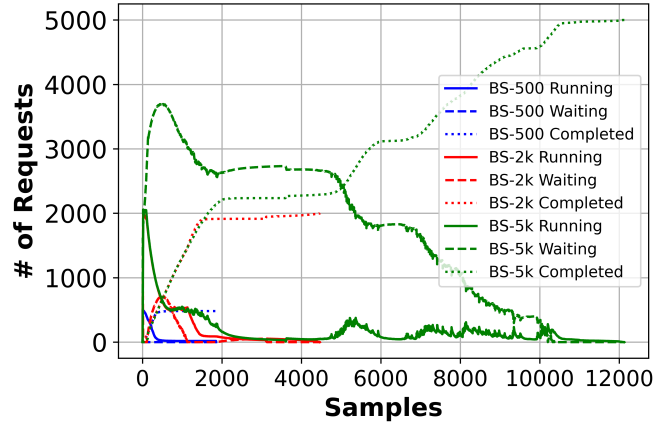


(b) GPU/HBM utilization for DeepSeek-R1-671B.

Fig. 13: Analysis of Prefill and Decode resource utilization for varying context lengths.



(a) KV Cache utilization.



(b) Request Analysis.

Fig. 14: Analysis of Prefill and Decode resource utilization of Llama405B for varying context lengths.

characterize this scaling behavior in Figure 15 for the 8B model and Figure 14 for the 405B model.

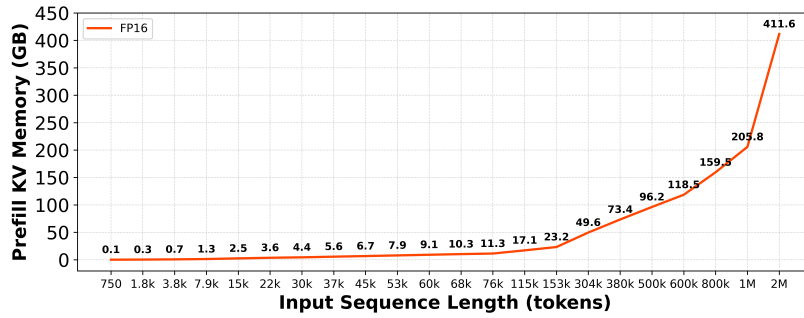
Observation 8: The “reasoning cliff” occurs when the growing KV cache exceeds available HBM capacity and forces the scheduler to preempt, recompute, or reject additional work. For short-output workloads, this cliff typically appears later in decode. For reasoning workloads with long outputs, KV growth pulls the cliff earlier, sometimes limiting admission during prefill. Systems should therefore estimate future KV growth at admission time and reserve HBM capacity for long-running decode phases instead of admitting requests based only on current memory usage.

Linearity vs. Capacity: Figure 15b demonstrates that the decode-phase KV footprint grows linearly with OSL. For the 8B model, a 20M token output (simulating aggregate reasoning

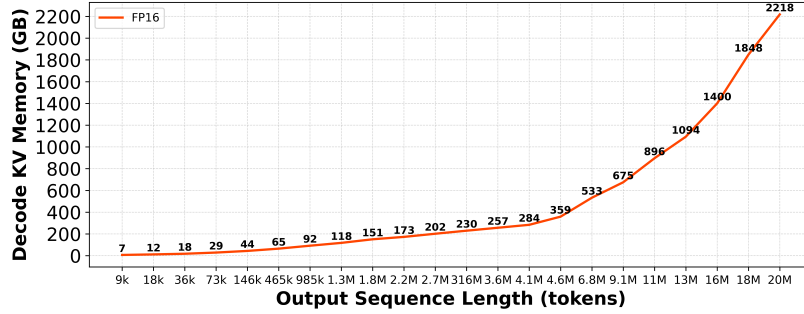
across a batch) demands over 2 TB of memory. While prefill KV also grows linearly (Figure 15a), it is transient. The decode KV is persistent. For the Llama-405B model (Figure 14a), this scaling creates a “Reasoning Cliff.” At a batch size of 1K with long context, the KV cache hits 100% utilization during the decode phase. However, as the batch size increases to 4K or 5K, the saturation point shifts *forward* in time as the cache exhausts during the prefill phase. This observation implies that for massive reasoning batches, the system cannot initialize requests before hitting the HBM wall.

C. System Mitigation: Chunked Prefills and Preemption

To survive the “Reasoning Cliff” mentioned above, the inference engine must dynamically degrade service, which introduces a negative dimension that is interesting to study. To this end, we analyze the vLLM scheduler’s response to this pressure using the Llama-405B request analysis, as depicted in Figure 14b.



(a) Prefill Phase KV Cache Utilization for varying ISL.



(b) Decode Phase KV Cache Utilization for varying OSL.

Fig. 15: Analysis of Prefill and Decode Memory Requirements during AI Inference.

Observation 9: For reasoning workloads, the scheduler must regulate the number and length of concurrent decode streams to keep KV usage within the HBM envelope. This makes scheduling a memory-traffic shaping problem: HBM capacity bounds sustainable throughput, while HBM bandwidth bounds per-token latency. Practical schedulers should therefore incorporate KV-aware admission control, decode throttling, and priority policies that balance TTFT, TPOT, and preemption risk.

Throttling Dynamics: When the batch size reaches 5K (green line), the system cannot fit the full state in memory. The scheduler activates **Chunked Prefill**, processing only a subset of the prompt tokens per iteration.

- **Queueing Behavior:** The “Running” curve (solid green) plateaus while the “Waiting” curve (dashed green) remains high. This indicates the system is serializing the admission of requests. Unlike the 500-batch case (blue), where requests enter and exit quickly, the 5K case forces the system into a “convoy” mode where new reasoning traces are only admitted as old ones finish and free their KV blocks.
- **Throughput-Latency Tradeoff:** By chunking prefills, the system prevents OOM crashes but introduces a “Start-Up Latency” for reasoning tasks. The GPU utilization remains high, but it is effectively stalling on memory capacity management rather than productive token generation.

VII. DISCUSSION

A forward-looking view of architecture design reveals a clear bifurcation between compute-intensive prefill and bandwidth-bound decode, motivating a decisive shift toward hardware disaggregation. Empirical characterizations underscore the fundamentally different bottlenecks of these phases, prompting next-generation systems to physically and logically decouple their execution paths to improve performance [15].

To meet the divergent requirements of large-scale inference, a tiered architecture that offloads the prefill phase to accelerators optimized for dense compute is optimal [19]. High-TFLOP devices with HBM having moderate bandwidth can deliver superior arithmetic throughput while providing sufficient capacity and bandwidth for prompt ingestion, as long as the memory subsystem can sustain the higher core utilization. When capacity or bandwidth becomes a constraint, the system can bank on offloading to system memory and utilize multiple tiers together with high-speed interconnects such as NVLink. This arrangement balances prefill throughput against the cost and constraints of the GPU-HBM complex, achieving an efficient trade-off between compute, capacity, and interconnect provisioning.

In contrast, the decode phase is governed by memory bandwidth and capacity, making it a better fit for scalable, memory-centric hierarchies. Combinations of HBM with CXL [7], [13] and near-memory compute solutions, including 3D-stacked memory (e.g., SRAM-based designs from D-Matrix [14]), alleviate the bandwidth pressure on KV reads. Operationally, decode benefits from explicit pooling and tiered offload across HBM → DDR/LP → CXL → NVMe, underpinned by fast

links such as NVLink or optical to preserve fleet-wide latency and throughput. 3D-stacked options that add DRAM atop HBM at the same pin speed can raise effective bandwidth to KV accesses without widening I/O, reducing the need to proliferate external HBM stacks. The principal trade-off is that bandwidth scales faster than usable per-stack capacity due to thermal limits, yield, and stack height. If decode continues to demand both higher bandwidth and larger KV stores, deeper 3D stacks primarily address the bandwidth side while capacity grows more slowly and at higher cost and complexity. Balanced designs therefore pair 3D bandwidth tiers with pooled DRAM/CXL/NVMe for capacity, tied together with fast optical interconnects. Software can exploit this bifurcation by reserving high-bandwidth resources exclusively for token generation while routing prefill to cost-optimized tiers, using advanced runtime strategies to maximize efficiency.

High-bandwidth flash (HBF) [32] approaches that augment HBM with a NAND tier are a potential fit for both phases. For prefill, HBF can stream model weights effectively; for decode, it offers higher capacity for colder KV entries and weights. However, HBF introduces challenges: higher power, read/write asymmetry, lower bandwidth, and substantial software uplift. Prefill’s mixed read/write behavior makes it sensitive to this asymmetry, potentially depressing throughput even as NAND capacity helps accommodate growing input contexts. For decode, the extra capacity is beneficial for long output sequence lengths and reasoning-heavy workloads, but the higher bandwidth demands necessitate intelligent KV management and caching. In practice, funneling traffic through the limited HBM channels in HBF can create bottlenecks unless the software stack tightly orchestrates placement, prefetching, and eviction.

Agentic AI further amplifies memory-system pressure by transforming inference from a single long reasoning trajectory into a stateful, multi-step execution spanning both accelerator and CPU domains. A request may invoke planner, executor, verifier, retrieval, and tool-augmented model calls, each propagating intermediate context, reasoning traces, and active KV state across iterations. As a result, the memory footprint scales not only with model size and CoT length, but also with agent fan-out, branching depth, tool interaction frequency, and concurrent sessions. Modern agentic architectures further rely on tightly coupled CPU–GPU execution, where CPUs sustain large numbers of independent agent environments, manage tool execution, and maintain per-agent state, queues, and retrieval data. Each agent incurs persistent DRAM overhead for context staging, tool outputs, sandbox execution, and vector data, placing host memory capacity and bandwidth directly on the execution path. This shifts the bottleneck from isolated GPU memory capacity to a system-wide problem: HBM must sustain high KV residency and reuse, while host DRAM actively participates in context management, data staging, and CPU-side processing. Compared with single-shot inference, agentic workloads create a multiplicative demand across HBM and DRAM, stressing both per-GPU limits and rack-level scheduling, motivating KV-aware placement, migration, and tiered memory management across HBM, host, CXL, and

storage tiers under strict TTFT, throughput, and ms/token SLAs.

Given sublinear performance scaling with model size and tight KV capacity constraints, future systems should move beyond monolithic, GPU-local KV caches and adopt explicit control over KV placement and migration. Offloading, pooling, and hierarchical tiering across HBM and host memory enable proactive eviction, compression, and reuse of pre-generated blocks, materially reducing time to first token. This memory flexibility allows heterogeneous deployments that combine small, distilled models for high-throughput speculative decoding with large foundation models for complex queries. To bind these disaggregated components, next-generation interconnects especially NVLink and optical links are essential across both prefill and decode paths, delivering the low-latency, high-throughput data movement required across tiers and between accelerators.

VIII. CONCLUSIONS

This work presented a systematic characterization of GPU-based inference for reasoning-centric LLMs, evaluating models from 8B to 671B parameters across diverse parallelization strategies. Our results highlight a fundamental divergence in scaling behavior: Data Parallelism achieves near-linear throughput for small models but faces hard capacity limits from KV-cache saturation, while Tensor Parallelism is essential for ultra-large models yet remains bottlenecked by memory bandwidth and synchronization costs. A key contribution of this study is the quantification of the “performance cliff” in reasoning workloads. We demonstrated that the extended context lengths required for multi-step reasoning exacerbate the disparity between the compute-bound prefill phase and the memory-bandwidth-bound decode phase. As sequence lengths grow, the KV cache aggressively consumes HBM capacity, leading to rapid throughput degradation and request throttling once memory limits are exceeded. These insights suggest that standard monolithic scaling is reaching its limits for reasoning-dense applications. Future inference systems must prioritize architectural disaggregation, effectively decoupling the prefill and decode phases to optimize for their respective compute and bandwidth requirements. Furthermore, realizing the full potential of frontier-scale models will require innovations in tiered memory management and hardware-software co-design to mitigate the severe capacity and bandwidth constraints identified in this analysis.

ACKNOWLEDGEMENTS

This work was supported in part by the U.S. Department of Energy under Contract DE-AC02-06CH11357, as well as the National Science Foundation (NSF), Office of Advanced Cyberinfrastructure, under grants CSSI-2411386 and CSSI-2514056.

REFERENCES

- [1] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov, “Vidur: A large-scale simulation framework for llm inference,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 351–366, 2024.

- [2] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 117–134.
- [3] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills," *arXiv preprint arXiv:2308.16369*, 2023.
- [4] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.
- [5] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "Llm in a flash: Efficient large language model inference with limited memory," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 12 562–12 584.
- [6] M. Arif, K. Assogba, M. M. Rafique, and S. Vazhkudai, "Exploiting cxl-based memory for distributed deep learning," in *Proceedings of the 51st International Conference on Parallel Processing*, ser. ICPP '22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3545008.3545054>
- [7] M. Arif, A. Maurya, and M. M. Rafique, "Accelerating performance of gpu-based workloads using cxl," in *Proceedings of the 13th Workshop on AI and Scientific Computing at Scale Using Flexible Computing*, ser. FlexScience '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 27–31. [Online]. Available: <https://doi.org/10.1145/3589013.3596678>
- [8] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "Moe-lightning: High-throughput moe inference on memory-constrained gpus," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 715–730.
- [9] Y. Cheng, Y. Liu, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, K. Du, and J. Jiang, "Lmcache: An efficient kv cache layer for enterprise-scale llm inference," *arXiv preprint arXiv:2510.09665*, 2025.
- [10] K. T. Chitty-Venkata, S. Raskar, B. Kale, F. Ferdous, A. Tanikanti, K. Raffanetti, V. Taylor, M. Emani, and V. Vishwanath, "Llm-inference-bench: Inference benchmarking of large language models on ai accelerators," in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1362–1379.
- [11] K. T. Chitty-Venkata, J. Ye, S. Raskar, A. Kougas, X. Sun, M. Emani, V. Vishwanath, and B. Nicolae, "PagedEviction: Structured block-wise KV cache pruning for efficient large language model inference," in *EACL 2026: 19th Conference of the European Chapter of the Association for Computational Linguistics*, Rabat, Morocco, 2026, pp. 3207–3218.
- [12] J.-B. Cordonnier, A. Loukas, and M. Jaggi, "Multi-head attention: Collaborate instead of concatenate," *arXiv preprint arXiv:2006.16362*, 2020.
- [13] CXL, "Compute express link," 2025, accessed: 2025-12-12. [Online]. Available: <https://computeexpresslink.org/>
- [14] D-Matrix, "Corsair™. built for generative ai," 2025, accessed: 2025-12-12. [Online]. Available: <https://www.d-matrix.ai/product/>
- [15] —, "Why we decoupled execution to accelerate i/o," 2025, accessed: 2025-12-12. [Online]. Available: <https://www.d-matrix.ai/why-we-decoupled-execution-to-accelerate-i-o/>
- [16] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Sun, Z. Zhu, M. Zhang, M. Cheng, S. Li, M. A. R. Bigas, Y. Hu, S. Zhu, and Z. Kuang, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025. [Online]. Available: <https://arxiv.org/abs/2501.12948>
- [17] Y. Dong, Y. Miao, W. Li, X. Zheng, C. Wang, J. Wu, and F. Lyu, "Accelerating LLM inference throughput via asynchronous KV cache prefetching," in *AAAI'26: The 2026 AAAI Conference on Artificial Intelligence*, vol. 40, no. 25, 2026, pp. 20 844–20 851.
- [18] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle *et al.*, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [19] Dynamo, "Nvidia dynamo," 2025, accessed: 2025-12-12. [Online]. Available: <https://developer.nvidia.com/dynamo>
- [20] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [21] M. Isaev, N. McDonald, L. Dennison, and R. Vuduc, "Calculon: a methodology and tool for high-level co-design of systems and large language models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–14.
- [22] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [23] B. Li, Y. Jiang, V. Gadepally, and D. Tiwari, "Llm inference serving: Survey of recent advances and opportunities," in *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2024, pp. 1–8.
- [24] H. Li, Y. Li, A. Tian, T. Tang, Z. Xu, X. Chen, N. Hu, W. Dong, Q. Li, and L. Chen, "A survey on large language model acceleration based on kv cache management," *arXiv preprint arXiv:2412.19442*, 2024.
- [25] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu *et al.*, "Deepseek-v3 technical report," 2024. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [26] A. Liu, J. Liu, Z. Pan, Y. He, G. Haffari, and B. Zhuang, "Minicache: Kv cache compression in depth dimension for large language models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 139 997–140 031, 2024.
- [27] A. Maurya, M. Rafique, F. Cappello, and B. Nicolae, "Mlp-offload: Multi-level, multi-path offloading for llm pre-training to break the gpu memory wall," in *SC'25: 38th International Conference for High Performance Computing, Networking, Storage and Analytics*, St Louis, USA, 2025.
- [28] OpenAI, "Openai o1 system card," 2024, accessed: 2025-02-12. [Online]. Available: <https://openai.com/index/openai-o1-system-card/>
- [29] S. Park, S. Jeon, C. Lee, S. Jeon, B.-S. Kim, and J. Lee, "A survey on inference engines for large language models: Perspectives on optimization and efficiency," *arXiv preprint arXiv:2505.01658*, 2025.
- [30] R. Qin, Z. Li, W. He, J. Cui, H. Tang, F. Ren, T. Ma, S. Cai, Y. Zhang, M. Zhang *et al.*, "Mooncake: A kv-cache-centric disaggregated architecture for llm serving," *ACM Transactions on Storage*, 2024.
- [31] S. Saareesitthipitak, A. Rao, C. Zhou, and W. Li, "Prophet: An llm inference engine optimized for head-of-line blocking," Stanford University, Technical Report (CS244B), 2024.
- [32] Sandisk, "Hbf: High bandwidth flash," 2025, accessed: 2025-12-12. [Online]. Available: <https://www.sandisk.com/company/newsroom/blogs/2025/memory-centric-ai-sandisks-high-bandwidth-flash-will-redefine-ai-infrastructure>
- [33] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.
- [34] Y. Wang and Z. Li, "Mechanistic interpretability of attention heads in reasoning llms," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, explains the "thinking" process at the attention layer level.
- [35] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [36] F. Xu, Q. Hao, Z. Zong, J. Wang, Y. Zhang, J. Wang, X. Lan, J. Gong, T. Ouyang, F. Meng *et al.*, "Towards large reasoning models: A survey of reinforced reasoning with large language models," *arXiv preprint arXiv:2501.09686*, 2025.
- [37] J. Ye, J. Cernuda, A. Maurya, X.-H. Sun, A. Kougas, and B. Nicolae, "Characterizing the behavior and impact of kv caching on transformer inferences under concurrency," in *IPDPS'25: The 39th IEEE International Parallel and Distributed Processing Symposium*, Milan, Italy, 2025. [Online]. Available: <https://hal.inria.fr/hal-04984000>
- [38] W. Yuan, J. Yu, S. Jiang, K. Padhe, Y. Li, I. Kulikov, K. Cho, D. Wang, Y. Tian, J. E. Weston *et al.*, "Naturalreasoning: Reasoning in the wild with 2.8 m challenging questions," *arXiv preprint arXiv:2502.13124*, 2025.
- [39] C. Zhao, C. Deng, C. Ruan, D. Dai, H. Gao, J. Li, L. Zhang, P. Huang, S. Zhou, S. Ma *et al.*, "Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1731–1745.